Volatility Memory Analysis — Step-by-Step Guide



A practical, hands-on walkthrough of memory forensics with **Volatility 3** on the CPIA lab. Work through the steps in order. Each section builds on the last, moving from setup to a full memory-triage investigation.

0. What you need and how this fits the lab

Volatility 3 is installed on the CPIA analyst box under the short command `vol`:

© Billy Forrest @ https://training.forrest.one

```
docker exec -it cpia-forensics bash
vol --version        # confirm it runs; you are inside /work
```

Memory analysis examines a **memory image** — a snapshot of a machine's RAM taken while it was running. It lets you recover running processes, loaded modules, network sockets, open files and (crucially) malware that hides only in memory. It is the definitive technique for finding **fileless malware** and in-memory implants that leave no trace on disk.

The CPIA lab does not bundle full multi-GB memory images (they are too large to ship in the Docker build). You download a public training image once on the host, then analyse it inside the analyst box. See step 1.

Concept Meaning Memory image (`.mem` / `.raw` / `.vmem`)	Byte-for-byte dump of RAM
Profile / symbol table	Map between raw bytes and Windows structures (Volatility 3 auto-detects)
Plugin	A single analysis (e.g. `windows.pslist` lists processes)
Running processes	Evidence of what was executing at capture time
Unbacked memory	Executable pages with no file on disk — a hallmark of injected code

---

1. Get a memory image (do this on the host)

The analyst box has **no internet egress**, so download the image on the host first, into the shared `/work` folder that the container can read.

Step 1.1 — pick an image. The Volatility project maintains a list of official training images (CCDC / Volatility memory samples):

```
https://github.com/volatilityfoundation/volatility3/wiki/Memory-Sample
```

Pick a **Windows** image (for example a `Win7SP1x64` or `Win10` sample) — these exercise the `windows.*` plugins used throughout this guide.

Step 1.2 — download it on the host into `work/`:

```
cd ~/labs/cpia-labs/work
curl -LO <memory-image-url>     # do this on the HOST, not in the container
```

Step 1.3 — confirm it is visible to the analyst box:

```
docker exec -it cpia-forensics bash
ls -lh /work                    # your image should appear here
```

> *Note: `/work` is the shared, persistent folder between the host and the analyst box. Anything you place there survives container restarts and is also browsable at `http://<host-ip>:8010/` on the host.*

---

2. First contact — confirm Volatility can read the image

Step 2.1 — basic information. Run the `info` plugin to confirm Volatility recognises the image and can auto-detect the OS:

```
cd /work
vol -f <image>.mem windows.info
```


© Billy Forrest @ https://training.forrest.one

You should see the OS version, kernel base address and the symbol table that Volatility auto-selected. If it fails here, the image may be corrupt or in an unsupported format — check the file type first:

```
file /work/<image>.mem
```

Step 2.2 — write down the key facts. Record the operating system, build number and capture context. These go in the header of your incident report.

---

3. Find running processes (start here every time)

Process enumeration is where every memory investigation begins: it shows what was executing when the machine was captured.

Step 3.1 — list processes three ways. Each plugin uses a different detection method, so comparing them catches hidden processes:

```
vol -f <image>.mem windows.pslist     # walks the active process list
vol -f <image>.mem windows.psscan     # scans physical memory for process objects
vol -f <image>.mem windows.pstree     # parent/child tree — shows the launch order
```

Step 3.2 — read the output. `windows.pslist` columns include PID, PPID, process name and start time. Look for:

- Processes that are **not** children of the normal Windows service manager (unusual PPIDs).
- Suspicious names or misspellings (`scvhost.exe`, `expl0rer.exe`).
- Processes with very recent start times clustered around the same moment (a dropped payload launching).

Step 3.3 — compare the lists. `psscan` finds process objects in raw memory even if they were unlinked from the active list. A process visible in `psscan` but **absent from** `pslist` is a strong indicator of process hiding / rootkit activity.

---

4. Correlate processes with network activity

A memory image captures the machine's network state too. Pair processes with sockets to link a suspicious process to command-and-control traffic.

```
vol -f <image>.mem windows.netstat    # active TCP/UDP connections
vol -f <image>.mem windows.netscan    # scan-based alternative (catches more)
```

Step 4.1 — identify the foreign connections. Look for established connections to external IPs on high or unusual ports, especially if a suspicious PID from step 3 owns them.

Step 4.2 — cross-reference. Match the PID from `netstat` back to the process name in `pslist`. A known-good process (e.g. `svchost.exe`) connecting to an unknown external IP is worth investigating; a suspicious process connecting out is usually the smoking gun.

---

5. Find injected code and malware (the core of memory triage)

The most valuable Volatility plugins detect code that exists **only in memory** — the signature of process injection and fileless malware.

Step 5.1 — scan for injected/executable memory:

© Billy Forrest @ https://training.forrest.one

```
vol -f <image>.mem windows.malfind
```

`malfind` looks for executable, **unbacked** memory regions (no corresponding file on disk) with `MZ` or `PAGE_EXECUTE_READWRITE` protections. Anything it flags should be treated as suspicious until proved otherwise.

Step 5.2 — confirm the process is carrying something odd:

```
vol -f <image>.mem windows.dlllist     # loaded DLLs per process
vol -f <image>.mem windows.ldrmodules  # detect hidden/unlinked modules
```

Compare the DLL list of a flagged process against the others. A process with modules that do not appear in its normal load set (or that are unlinked per `ldrmodules`) is a strong injection signal.

Step 5.3 — dump the suspicious memory for closer inspection:

```
vol -f <image>.mem -o /work/dumps windows.dumpfiles --pid <PID>
```

The `-o /work/dumps` flag writes extracted files to a folder you can then inspect (and later scan with the malware-triage tools — see step 8).

---

6. Recover commands, files and credentials

Volatility can also recover what the user (or attacker) actually did, plus sensitive data left in memory.

Step 6.1 — recover command lines and console input:

```
vol -f <image>.mem windows.cmdline     # full command lines of all processes
vol -f <image>.mem windows.consoles    # commands typed into open consoles
vol -f <image>.mem windows.envars      # environment variables per process
```

`cmdline` frequently exposes the exact malicious command an attacker ran. `consoles` can recover interactive commands typed at a shell.

Step 6.2 — recover open files:

```
vol -f <image>.mem windows.filescan    # files present in memory cache
```

`filescan` shows files the OS had open or cached. Look for downloaded payloads in `%TEMP%`, `Downloads` or `AppData` that you can then dump and analyse.

Step 6.3 — dump hashes (if investigating credential access):

```
vol -f <image>.mem windows.hashdump    # local account password hashes
vol -f <image>.mem windows.cachedump   # cached domain credentials
vol -f <image>.mem windows.lsadump     # LSA secrets
```

> *Use these with care and only in a sanctioned lab or investigation. They are included here because credential dumping is a common intrusion-analyst task, but handle recovered credentials as sensitive data.*

---

7. Extract a timeline and the registry view

Step 7.1 — build a memory timeline:

```
vol -f <image>.mem windows.timeliner
```

`timeliner` gathers timestamps from processes, files and other artefacts into one chronological view. Use it to reconstruct the order of events: when the dropper ran, when the beacon first

© Billy Forrest @ https://training.forrest.one

phoned home, when persistence was planted.

Step 7.2 — inspect the registry (persistence hunting):

```
vol -f <image>.mem windows.registry.hivelist        # locate registry hives
vol -f <image>.mem windows.registry.printkey --key "Software\Microsoft\Windows\CurrentVersion\Run"
vol -f <image>.mem windows.registry.scheduled_tasks # scheduled tasks
```

Autorun keys and scheduled tasks are the classic persistence mechanisms. Check `Run` keys and the `RunOnce` equivalents, and compare against a known-good baseline for the OS version.

---

8. Document your findings (the CPIA way)

The analyst box includes malware-triage tooling for follow-up on anything you dump (step 5.3). Once you have a file, identify and hash it:

```
file /work/dumps/<file>
sha256sum /work/dumps/<file>
strings -n 8 /work/dumps/<file> | head -50
```

Then record the investigation. For each artefact, capture:

Field Example Image / capture	`Win7SP1x64-sample.mem`
Plugin used	`windows.malfind`
PID / process	`1024 svchost.exe`
Indicator	unbacked executable region in PID 1024
IOC (hash/IP/domain)	`sha256:ab12…`
Timeline position	14:02:31 — after initial access
Conclusion	injected Cobalt Strike beacon

Step 8.1 — write the report. State the verdict (compromised or not), the evidence chain (process → network → injected code → persistence) and the IOCs. A memory analysis is only as strong as its documentation.

---

9. Quick reference — the commands at a glance

```
# Inside the analyst box, from /work:
vol -f <image>.mem windows.info                    # confirm the image parses
vol -f <image>.mem windows.pslist                  # active processes
vol -f <image>.mem windows.psscan                  # scan-based process list
vol -f <image>.mem windows.pstree                  # parent/child tree
vol -f <image>.mem windows.netstat                 # active connections
vol -f <image>.mem windows.netscan                 # scan-based sockets
vol -f <image>.mem windows.malfind                 # injected/unbacked code
vol -f <image>.mem windows.dlllist                 # loaded DLLs
vol -f <image>.mem windows.ldrmodules              # hidden modules
vol -f <image>.mem windows.cmdline                 # command lines
vol -f <image>.mem windows.consoles                # console input
vol -f <image>.mem windows.filescan                # cached/open files
vol -f <image>.mem windows.timeliner               # full timeline
vol -f <image>.mem -o /work/dumps windows.dumpfiles --pid <PID>
vol -f <image>.mem windows.registry.hivelist
vol -f <image>.mem windows.registry.printkey --key "Software\Microsoft\Windows\CurrentVersion\Run"
```

A typical workflow in one pass:

© Billy Forrest @ https://training.forrest.one

```
vol -f <image>.mem windows.pstree     # spot the odd process
vol -f <image>.mem windows.malfind    # confirm injected code
vol -f <image>.mem windows.netscan    # link it to a C2 socket
vol -f <image>.mem -o /work/dumps windows.dumpfiles --pid <PID>
# then: file + sha256sum + strings on the dump
```

10. Troubleshooting

Symptom Cause / fix `vol` not found	You are on the host. Enter the box: `docker exec -it cpia-forensics bash`
`file <img>` shows wrong type	Image not a raw dump; convert or use `--single-location` / correct format
`windows.info` errors	Unsupported/truncated image; re-download it on the host
No `windows.*` plugins	Image is Linux/macOS — use `linux.*` / `mac.*` plugin families instead
Empty `malfind` output	Good news — no obvious unbacked executable memory; keep checking `psscan` vs `pslist`
Dumps folder missing	Create it first: `mkdir -p /work/dumps`

© Billy Forrest

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one